



The OpenVX[™] Tensor from Image Extension

The Khronos[®] OpenVX Working Group, Contributor: Steve Ramm

Version 1.3.1, Tue, 29 Apr 2025 13:24:58 +0000: Git branch information not available

Table of Contents

1. Extension to support tensor views of images	2
1.1. Purpose	2
1.2. Example Use Cases	2
1.3. Acknowledgements	2
2. Requirements	4
2.1. Functions	4
2.1.1. vxCreateTensorFromROI	4
2.1.2. vxCreateTensorFromChannel	6
2.1.3. vxCreateTensorObjectArrayFromROI	7
2.1.4. vxCreateTensorObjectArrayFromChannel	8
2.1.5. vxCreateTensorObjectArrayFromView	10



Copyright 2013-2025 The Khronos Group Inc.

This specification is protected by copyright laws and contains material proprietary to Khronos. Except as described by these terms, it or any components may not be reproduced, republished, distributed, transmitted, displayed, broadcast or otherwise exploited in any manner without the express prior written permission of Khronos.

This specification has been created under the Khronos Intellectual Property Rights Policy, which is Attachment A of the Khronos Group Membership Agreement available at www.khronos.org/files/member_agreement.pdf. Khronos Group grants a conditional copyright license to use and reproduce the unmodified specification for any purpose, without fee or royalty, EXCEPT no licenses to any patent, trademark or other intellectual property rights are granted under these terms. Parties desiring to implement the specification and make use of Khronos trademarks in relation to that implementation, and receive reciprocal patent license protection under the Khronos IP Policy must become Adopters and confirm the implementation as conformant under the process defined by Khronos for this specification; see <https://www.khronos.org/adopters>.

Khronos makes no, and expressly disclaims any, representations or warranties, express or implied, regarding this specification, including, without limitation: merchantability, fitness for a particular purpose, non-infringement of any intellectual property, correctness, accuracy, completeness, timeliness, and reliability. Under no circumstances will Khronos, or any of its Promoters, Contributors or Members, or their respective partners, officers, directors, employees, agents or representatives be liable for any damages, whether direct, indirect, special or consequential damages for lost revenues, lost profits, or otherwise, arising from or in connection with these materials.

Khronos is a registered trademark, and OpenVX is a trademark of The Khronos Group Inc. OpenCL is a trademark of Apple Inc., used under license by Khronos. All other product names, trademarks, and/or company names are used solely for identification and belong to their respective owners.

Chapter 1. Extension to support tensor views of images

1.1. Purpose

Visual perception often requires that images are processed as tensors, which may be inconvenient if they have previously be processed as images, for example to correct for aberrations or extract a channel from a multi-channel image. As always, there is a desire to eliminate copies wherever possible, so rather than copy data from an image to a tensor it would be better if the tensor handled the data as a 'view' of the original image, i.e. the tensor uses exactly the same data locations as the image, and when one is changed it affects the other. Because of the single-plane nature of tensors, this operation can only be performed with single-plane images, or at least one plane of an image, and the type of that image plane will determine the type of the tensor.

Additionally, node replication is frequently use to process several images in parallel, hence it's also necessary to have object arrays of tensors where each tensor is a view of an image in a corresponding object array of images.

For convenience we supply functions to:

- Create a tensor view of an entire single-plane image or a subset (ROI) of a single-plane image.
- Create a tensor view of a single channel of a multi-channel image; the implementation is only required to support the case where a channel occupies the entire plane.
- Create object arrays of tensors as views of object arrays of single-plane images (entire or ROI).
- Create object arrays of tensors as views of single channels of object arrays of multi-channel images.
- For completeness, a function to create an object array of tensors as views of an object array of tensors.

Note that an entire plane of various single-plane interleaved images types may be mapped to a tensor of type `VX_TYPE_UINT8` but the effect is dependent upon the underlying pixel format. The dimensions of the output tensor do not necessarily correspond to the dimensions of the input image in that the height, width or both may be changed according to the actual number of bytes used to store the pixel values. For example an image of packed 1-bit pixels will result in a tensor of width one-eighth if the image where each element consists of 8 pixels, and images of type RGB and RGBX will result in tensors where the width is four times the width of the image, and the pixels are interleaved. For sub-sampled formats, the tensor dimensions will be less than that of the image. In theses cases, the tensor view of the image can be thought of as a "plain old data" view.

Note that as tensors support fixed-point arithmetic, the ability to specify a fixed-point position in the tensor view of the image is supplied.

1.2. Example Use Cases

- Processing of images by nodes that expect tensors
- Replication of nodes in graphs where tensor views of images are involved

1.3. Acknowledgements

This specification would not be possible without the contributions from this partial list of the following individuals from the Khronos Working Group and the companies that they represented at the time:

- Simon Barfield - ETAS (Robert Bosch GmbH)
- Raphael Cano - Robert Bosch GmbH
- Radhakrishna Giduthuri - Intel
- Andrew Graves - ETAS (Robert Bosch GmbH)
- Viktor Gyenes - AI Motive
- Kiriti Nagesh Gowda - AMD
- Stephen Ramm - ETAS (Robert Bosch GmbH)
- Jesse Villarreal - TI

Chapter 2. Requirements

2.1. Functions

2.1.1. vxCreateTensorFromROI

[REQ-TFI01]

Creates a Tensor from a region of interest in a virtual or non-virtual image.

Function signature:

```
vx_tensor vxCreateTensorFromROI(vx_image image, const vx_rectangle_t *rect, vx_int8 fixed_point_position);
```

Parameters

- [in] **image** - The reference to the parent image.
- [in] **rect** - Pointer to the region of interest rectangle.
- [in] **fixed_point_position** - Specifies the fixed point position. If 0, calculations are performed using integer arithmetic.

[REQ-TFI02]

The input **image** must be a non-virtual or virtual single-plane image with width, height and format all defined.

[REQ-TFI03]

The input pointer **rect** must either be NULL or point to a `vx_rectangle_t` defining points within the parent image pixel space; if the pointer is NULL then the bounds shall be set equal to the entire image.

[REQ-TFI04]

For single-plane images of a simple numeric image format (DF_IMAGE_U8, _U16, _U32, _S16, _S32) the tensor shall be created with the dimensions given by **rect** and the corresponding numeric type (VX_TYPE_UINT8, UINT16, UINT32, INT16, INT32).

[REQ-TFI05]

For single-plane images of format DF_IMAGE_U1, the tensor shall be created with a type of VX_TYPE_UINT8, a first dimension equal to $(\text{width} + 7) / 8$, and the second dimension equal to the height, the width and height being defined by **rect**. Note that for this format the origin of the **rect** must be at a multiple of 8 pixels, due to the packed nature of the format.

[REQ-TFI06]

For other interleaved single-plane formats including sub-sampled formats, tensors shall be created as follows:

- VX_DF_IMAGE_RGB: tensor type VX_TYPE_UINT32 with padding in the high order byte.
- VX_DF_IMAGE_RGBX: tensor type VX_TYPE_UINT32.

- `VX_DF_IMAGE_YUYV`, `VX_DF_IMAGE_UYVY`: tensor type `VX_TYPE_UINT16`.

Padding bits are unchanged from the original image data.

The following table summarises the previous requirements:

Table 1. Image formats, tensor dimensions and tensor types

Image format	Tensor dimension 1	Tensor dimension 2	Tensor type	Notes
<code>DF_IMAGE_U8</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT8</code>	
<code>DF_IMAGE_U16</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT16</code>	
<code>DF_IMAGE_U32</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT32</code>	
<code>DF_IMAGE_S16</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_INT16</code>	
<code>DF_IMAGE_S32</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_INT32</code>	
<code>DF_IMAGE_U1</code>	$(\text{rect.endx} - \text{rect.startx} + 7) / 8$	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT8</code>	$(\text{rect.startx} \% 8) == 0$
<code>DF_IMAGE_RGB</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT32</code>	High order byte of each tensor element is padding
<code>DF_IMAGE_RGBX</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT32</code>	
<code>DF_IMAGE_YUYV</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT16</code>	Entries alternate as YU, YV
<code>DF_IMAGE_UYVY</code>	<code>rect.endx - rect.startx</code>	<code>rect.endy - rect.starty</code>	<code>VX_TYPE_UINT16</code>	Entries alternate as UY, VY

Returns

[REQ-TFI07]

A reference to the tensor; any possible errors preventing a successful creation may be checked using `vxGetStatus()`.

Possible causes of errors are:

- Invalid input reference
- Input reference is not to an image
- Input image does not have its width, height or format specified
- The values in the rectangle do not fall within the image bounds
- Out of resources

[REQ-TFI08]

The new reference refers to data in the original image, so that updates to the tensor shall update the parent image, and updates to the ROI in the parent image shall update the tensor.

[REQ-TFI09]

If the input image is virtual, the new reference returned is to a virtual tensor. If the input image is non-virtual, the

new reference returned is to a non-virtual tensor.

2.1.2. vxCreateTensorFromChannel

[REQ-TFI10]

Creates a tensor or virtual tensor from a single plane channel of another image. The original image may be virtual or non-virtual.

Function signature:

```
vx_tensor vxCreateTensorFromChannel(vx_image image, vx_enum channel, vx_int8 fixed_point_position);
```

Parameters

- [in] **image** - The reference to the parent image
- [in] **channel** - The vx_channel to use.
- [in] **fixed_point_position** - Specifies the fixed point position. If 0, calculations are performed using integer arithmetic.

[REQ-TFI11]

The input **image** must be a non-virtual or virtual image with defined format.

[REQ-TFI12]

The function supports channels that occupy an entire plane of multi-planar images in **image_array**. Image formats and channels shall be supported as described in the following table:

Table 2. Image formats, supported channels and corresponding tensor type

Supported image format	Supported channel(s)	Tensor type
NV12	VX_CHANNEL_Y	VX_TYPE_UINT8
NV21	VX_CHANNEL_Y	VX_TYPE_UINT8
IYUV	VX_CHANNEL_Y	VX_TYPE_UINT8
	VX_CHANNEL_U	VX_TYPE_UINT8
	VX_CHANNEL_V	VX_TYPE_UINT8
YUV4	VX_CHANNEL_Y	VX_TYPE_UINT8
	VX_CHANNEL_U	VX_TYPE_UINT8
	VX_CHANNEL_V	VX_TYPE_UINT8

Returns

[REQ-TFI13]

A reference to the tensor; any possible errors preventing a successful creation may be checked using vxGetStatus().

Possible causes of errors are:

- Invalid input reference
- Input reference is not to an image
- The input image does not have its format specified
- Input image is not in a supported multi-planar format
- **channel** is not a valid channel comprising the entire plane of the input format
- Out of resources

[REQ-TFI14]

The new reference refers to data in the original image, so that updates to the new tensor update the parent image, and updates to the specified channel of the parent image update the tensor.

[REQ-TFI15]

If the input image is virtual, the new reference returned is to a virtual tensor. If the input image is non-virtual, the new reference returned is to a non-virtual tensor.

2.1.3. vxCreateTensorObjectArrayFromROI

[REQ-TFI16]

Creates an object array or virtual object array of tensors from another object array of images given a rectangle. The original object array may be virtual or non-virtual.

Function signature:

```
vx_object_array vxCreateTensorObjectArrayFromROI(vx_object_array image_array, const vx_rectangle_t* rect,
vx_int8 fixed_point_position);
```

Parameters

- [in] **image_array** - The reference to the parent object array
- [in] **rect** - The region of interest rectangle. Must contain points within the parent images' pixel space.
- [in] **fixed_point_position** - Specifies the fixed point position. If 0, calculations are performed using integer arithmetic.

[REQ-TFI17]

The input object array **image_array** must be a non-virtual or virtual object array of images with width, height and format all defined.

[REQ-TFI18]

The input pointer **rect** must either be NULL or point to a vx_rectangle_t defining points within the parent image pixel space; if the pointer is NULL then the bounds shall be set equal to the entire image.

[REQ-TFI19]

For single-plane images of a simple numeric image format (DF_IMAGE_U8, _U16, _U32, _S16, _S32) the tensor shall

be created with the same dimensions and the corresponding numeric type (VX_TYPE_UINT8, _UINT16, _UINT32, _INT16, _INT32), just as described for [vxCreateTensorFromROI](#).

[REQ-TFI20]

For single-plane images of format DF_IMAGE_U1, the tensors shall be created with a type of VX_TYPE_UINT8, a first dimension equal to $(\text{the image width} + 7) / 8$, and the second dimension equal to the height, just as described for [vxCreateTensorFromROI](#).

[REQ-TFI21]

For other interleaved single-plane formats including sub-sampled formats, tensors shall be created as described for [vxCreateTensorFromROI](#)

For a summary, please refer to [Image formats, tensor dimensions and tensor types](#).

Returns

[REQ-TFI22]

A reference to the array of tensors; any possible errors preventing a successful creation may be checked using `vxGetStatus()`.

Possible causes of errors are:

- Invalid input reference
- Input reference is not to an object array
- Input object array does not contain images
- Input images do not have their width, height or format specified
- Input object array images bounds are not defined, do not contain the region described by **rect** or are not supported for the image format
- Out of resources

[REQ-TFI23]

The new reference refers to data in the original array, so that updates to the tensors shall update the parent images, and updates to the parent images in the region of interest shall update the tensors.

[REQ-TFI24]

If the input object array is virtual, the new reference returned is to a virtual object array of tensors. If the input object array is non-virtual, the new reference returned is to a non-virtual object array of tensors.

2.1.4. vxCreateTensorObjectArrayFromChannel

[REQ-TFI25]

Creates an object array or virtual object array of tensors from a single plane channel of another object array of images. The original object array may be virtual or non-virtual.

Function signature:

```
vx_object_array vxCreateTensorObjectArrayFromChannel(vx_object_array image_array, vx_enum channel, vx_int8
fixed_point_position);
```

Parameters

- [in] **image_array** - The reference to the parent object array
- [in] **channel** - The vx_channel to use.
- [in] **fixed_point_position** - Specifies the fixed point position. If 0, calculations are performed using integer arithmetic.

[REQ-TFI26]

The input object array **image_array** must be a non-virtual or virtual object array of images with defined format.

[REQ-TFI27]

The function supports channels that occupy an entire plane of the multi-planar images in **image_array**. Formats and channels shall be supported as described for [vxCreateTensorFromChannel](#) in the table [Image formats, supported channels and corresponding tensor type](#).

Returns

[REQ-TFI28]

A reference to the array of tensors; any possible errors preventing a successful creation may be checked using `vxGetStatus()`.

Possible causes of errors are:

- Invalid input reference
- Input reference is not to an object array
- Input object array does not contain images
- Input images do not have their format specified
- Input images are not in a supported multi-planar format
- **channel** is not a valid channel comprising the entire plane of the input format
- Out of resources

[REQ-TFI29]

The new reference refers to data in the original array, so that updates to the tensors shall update the parent images, and updates to the specified channel of the parent images shall update the tensors.

[REQ-TFI30]

If the input object array is virtual, the new reference returned is to a virtual object array of tensors. If the input object array is non-virtual, the new reference returned is to a non-virtual object array of tensors.

2.1.5. vxCreateTensorObjectArrayFromView

[REQ-TFI31]

Creates an object array or virtual object array of tensors from another object array of tensors given lists of start and end points per dimension. The original object array may be virtual or non-virtual.

Function signature:

```
vx_object_array vxCreateTensorObjectArrayFromView(vx_object_array tensor_array, vx_size number_of_dims,  
const vx_size* view_start, const vx_size* view_end);
```

Parameters

- [in] **tensor_array** - The reference to the parent object array
- [in] **number_of_dims** - Number of dimensions in the view. Error return if 0 or greater than number of tensor dimensions. If smaller than number of tensor dimensions, the lower dimensions are assumed.
- [in] **view_start** - View start coordinates.
- [in] **view_end** - View end coordinates.

[REQ-TFI32]

The input object array **tensor_array** must be a non-virtual or virtual object array of tensors with dimensions and type all defined.

[REQ-TFI33]

The input pointers **view_start** and **view_end** must either be both NULL or point to arrays of length **number_of_dims** defining start and end points within the parent tensor; if the pointers are NULL then the bounds shall be set equal to zero and the size of each dimension. The end point must never be less than the start point; the size of each dimension is given by (end point) - (start point).

Returns

[REQ-TFI34]

A reference to the array of tensors; any possible errors preventing a successful creation may be checked using vxGetStatus().

Possible causes of errors are:

- Invalid input reference
- Input reference is not to an object array
- Input object array does not contain tensors
- Input tensors do not have their type or dimensions all defined
- Input object array tensors do not contain the region described by **view_start** and **view_end** or end points are less than start points
- Out of resources

[REQ-TFI35]

The new reference refers to data in the original array, so that updates to the new tensors shall update the parent tensors, and updates to the parent tensors in the region of interest shall update the new tensors.

[REQ-TFI36]

If the input object array is virtual, the new reference returned is to a virtual object array of tensors. If the input object array is non-virtual, the new reference returned is to a non-virtual object array of tensors.